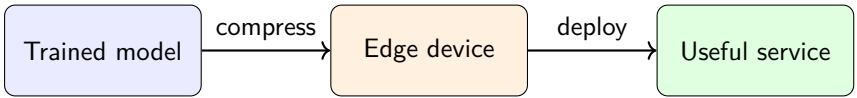
# Model Compression: Making Neural Networks Deployable

**Accurate is not enough.**




**Today's question**

How can we make a model smaller or cheaper *without losing too much useful behavior*?

# How Much Memory Does a Neural Network Need?

## Consider a model with 1 billion parameters

Each parameter requires storage determined by its numerical representation.

| Representation | Bytes / weight | Total size |
|---|---|---|
| FP32 | 4 | 4 GB |
| FP16 | 2 | 2 GB |
| INT8 | 1 | 1 GB |
| INT4 | 0.5 | 0.5 GB |

## The deployment problem

What if our target device has only 2 GB of available RAM?

Idealized weight storage only; excludes activations, temporary buffers, and representation metadata.

# Compression Ratio Is Not a Speedup

## Four different quantities

- **Parameters**: number of weights
- **Bytes**: storage and traffic
- **MACs/FLOPs**: arithmetic work
- **Latency**: elapsed time on device

## Numerical counterexample

100 MB $\rightarrow$ 25 MB

$4\times$ smaller weights $\not\Rightarrow$ $4\times$ faster inference.

Kernel overhead, activations and unsupported formats may dominate.

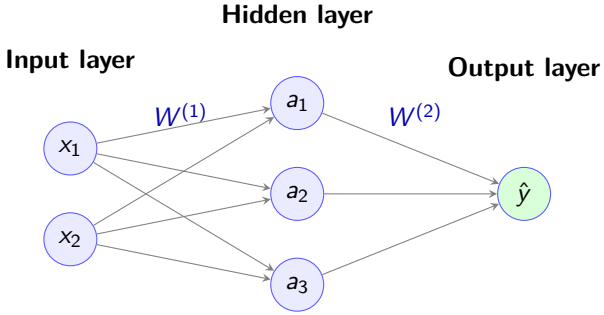
Measure the resulting executable, not only the representation.

# Compression and Execution Outcomes

| Metric | Original | Scenario A | Scenario B | Scenario C |
|---|---|---|---|---|
| Model size | 400 MB | 100 MB | 100 MB | 100 MB |
| Inference latency | 40 ms | 40 ms | 20 ms | 10 ms |
| Accuracy | 95% | 95% | 95% | 93% |
| Storage reduction | — | 4× | 4× | 4× |
| Execution speedup | — | 1× | 2× | 4× |

Compression does not mean making the model four times faster. Compression means reducing the resources required to represent or execute a model while preserving acceptable predictive quality. Depending on the technique and hardware, the benefit may be smaller storage, reduced memory usage, lower computation, faster inference, or some combination of these.

# A Neural Network Computes a Prediction




Intermediate activations

## What is learned?

- Weights $W$ and biases $b$
- Stored with the trained model

## What is computed?

- Activations for each input
- Intermediate numerical values

# A Tiny Numerical Forward Pass

## One neuron, no activation function

$$x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad w = \begin{bmatrix} 3 \\ -1 \end{bmatrix}, \quad b = 1$$

$$y = w^T x + b = (3)(2) + (-1)(1) + 1 = 6$$

## Change one weight

Set $w_2 \leftarrow 0$ (prune):

$$y' = (3)(2) + (0)(1) + 1 = 7$$

**Prediction changed:** $6 \rightarrow 7$.

Removing a weight changes the function, even if its shape is unchanged.

# Loss: A Score for Prediction Error

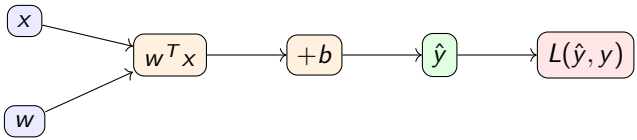
## Example: squared-error loss

$$L = (\hat{y} - y)^2$$

| Case | Target $y$ | Prediction $\hat{y}$ | Loss |
|---|---|---|---|
| Before modification | 6 | 6 | 0 |
| After pruning | 6 | 7 | 1 |
| After adjustment | 6 | 6.2 | 0.04 |

Loss is a *score*; an optimizer is the process that changes weights to reduce it.

# Computational Graphs: Calculations as Nodes




- Forward direction: compute a prediction and loss.
- Training: calculate how the loss responds to weight changes.
- **Backpropagation** efficiently computes that information;

# Gradients and Training: The Intuition

## Conceptual update

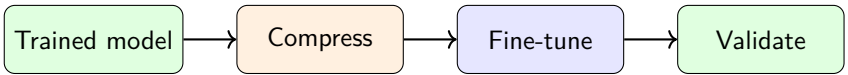
$$W_{\text{new}} = W_{\text{old}} - \eta \nabla_W L$$

- Gradient: how loss changes under small weight changes.
- Learning rate $\eta$: update step size.
- Repeated updates can decrease training loss.

## Today's boundary

Interpret the update; **do not** derive the gradients.

# Why Fine-Tuning Can Repair Compression Damage




- Compression perturbs weights or restricts model capacity.
- Remaining weights can sometimes adapt to restore quality.
- Measure the accuracy of the final model on held-out data.

1. Fine-tuning is a type of optimization, not a different optimization algorithm.
2. Fine-tuning is not defined by loss already being low.

It means continuing training from existing pretrained weights, often for a more specific objective or to adapt to a modification.

# Why Compression May Require Fine-Tuning

## Our simple neural network

$$\hat{y} = w_1 x_1 + w_2 x_2 \qquad x_1 = 2, \quad x_2 = 3, \quad y = 1.6$$

Loss function: $L = (\hat{y} - y)^2$

# Why Compression May Require Fine-Tuning

## Our simple neural network

$$\hat{y} = w_1 x_1 + w_2 x_2 \qquad x_1 = 2, \quad x_2 = 3, \quad y = 1.6$$

Loss function: $L = (\hat{y} - y)^2$

## 1. Trained Model

$$w_1 = 0.5, \quad w_2 = 0.2$$

$$\hat{y} = 0.5(2) + 0.2(3)$$

$$\hat{y} = 1.6$$

$$L = 0$$

# Why Compression May Require Fine-Tuning

## Our simple neural network

$$\hat{y} = w_1 x_1 + w_2 x_2 \qquad x_1 = 2, \quad x_2 = 3, \quad y = 1.6$$

Loss function: $L = (\hat{y} - y)^2$

## 1. Trained Model

$w_1 = 0.5, \quad w_2 = 0.2$

$\hat{y} = 0.5(2) + 0.2(3)$

$\hat{y} = 1.6$

$L = 0$

## 2. After Pruning

$w_1 = 0.5, \quad w_2 = 0$

$\hat{y} = 0.5(2) + 0(3)$

$\hat{y} = 1.0$

$L = 0.36$

# Why Compression May Require Fine-Tuning

## Our simple neural network

$$\hat{y} = w_1 x_1 + w_2 x_2 \qquad x_1 = 2, \quad x_2 = 3, \quad y = 1.6$$

Loss function: $L = (\hat{y} - y)^2$

### 1. Trained Model

$w_1 = 0.5, \quad w_2 = 0.2$

$\hat{y} = 0.5(2) + 0.2(3)$

$\hat{y} = 1.6$

$L = 0$

### 2. After Pruning

$w_1 = 0.5, \quad w_2 = 0$

$\hat{y} = 0.5(2) + 0(3)$

$\hat{y} = 1.0$

$L = 0.36$

### 3. After Fine-Tuning

$w_1 = 0.8, \quad w_2 = 0$

$\hat{y} = 0.8(2) + 0(3)$

$\hat{y} = 1.6$

$L = 0$

**Fine-tuning adjusts remaining weights to recover performance after compression.**

# Why Reduce Numerical Precision?

## Start with just one trained weight: $w = 0.25$

The weight is a number. Its *storage cost* depends on how we represent it.

| Representation | Raw space | What happens to 0.25? |
|---|---|---|
| FP32 | 32 bits = 4 bytes | Store as a floating-point number |
| FP16 | 16 bits = 2 bytes | Store as a shorter floating-point number |
| INT8 | 8 bits = 1 byte | Store integer 25 with scale $s = 0.01$ |

## Two questions to keep separate

**Memory:** How many bits do the weights occupy?
**Accuracy:** How closely do the new values reproduce the original predictions?

INT8 example assumes $q = \mathrm{round}(w/s)$; scales also require storage. Less memory does not guarantee lower latency.

# Floating Point vs Integer Quantization

## FP32 → FP16 / BF16

- Still a **floating-point number**.
- Standard sign, exponent and fraction encoding.
- 0.25 can be represented exactly in all three.
- Less precision/range may affect other values.

## FP32 → INT8 / INT4

- Store a **small integer code**.
- Need a **scale** to interpret that code.
- Example: $0.25 \leftrightarrow 25$ if $s = 0.01$.
- Fewer codes can cause rounding or clipping.

| Format | FP32 | FP16 | BF16 | INT8 | INT4 |
|---|---|---|---|---|---|
| Bits/value | 32 | 16 | 16 | 8 | 4 |

## Remember

Floating-point conversion changes the floating-point format; integer quantization uses an explicit code-to-value mapping.

# How Much Memory Can We Save?

## Scale up from one weight to $N = 100$ million weights

$$\text{Ideal weight bytes} = N \times \frac{\text{bits per weight}}{8}$$

| Format | Bytes/weight | Raw weight storage | vs FP32 |
|---|---|---|---|
| FP32 | 4 | 400 MB | baseline |
| FP16 / BF16 | 2 | 200 MB | 2× smaller |
| INT8 | 1 | 100 MB | 4× smaller |
| INT4 | 0.5 | 50 MB | 8× smaller |

Storage reduction $\neq$ equal-factor execution speedup.

Decimal MB; excludes biases (if not counted in $N$), quantization scales, packing overhead, activations and temporary memory.

# Quantization: Convert Real Weights to Integer Codes

## Our four trained weights: $W = [0.25, -0.50, 0.75, 1.00]$

Choose a symmetric INT8 scale $s = 0.01$:

$$q = \mathrm{round}(x/s), \qquad \hat{x} = s\,q$$

| Original $x$ | $x/0.01$ | Stored code $q$ | Recovered $\hat{x}$ |
|---|---|---|---|
| 0.25 | 25 | 25 | 0.25 |
| $-0.50$ | $-50$ | $-50$ | $-0.50$ |
| 0.75 | 75 | 75 | 0.75 |
| 1.00 | 100 | 100 | 1.00 |

## Why is this a mapping?

Code 25 means 0.25 *only when the scale is* 0.01. Other scales can introduce rounding error.

# Scale, Zero-Point and Clipping: One More Example

## Use a toy signed 4-bit quantizer: codes $-8$ through 7

Let scale $s = 0.25$ and zero-point $z = -1$:

$$q = \mathrm{clip}(\mathrm{round}(x/s) + z, -8, 7), \qquad \hat{x} = s(q - z)$$

| Original $x$ | Before clipping | Stored $q$ | Recovered $\hat{x}$ |
|---|---|---|---|
| 0.00 | $-1$ | $-1$ | 0.00 |
| 0.50 | 1 | 1 | 0.50 |
| 2.50 | 9 | 7 | 2.00 |

## Read the example

**Scale** gives the step size; **zero-point** encodes real zero; **clipping** prevents invalid codes but can lose information.

# Symmetric vs Asymmetric Quantization

## Symmetric

$$z = 0$$

- Convenient for roughly zero-centered weights
- Simpler integer math in many kernels

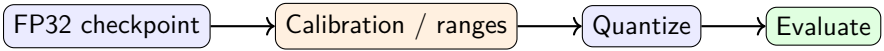
## Asymmetric

$$z \neq 0 \text{ (typically)}$$

- Shifts representable range
- Useful for nonnegative or skewed activations

Choice depends on data distribution and kernel support.

# PTQ: Compress an Already-Trained Model




- No weight retraining required.
- Weight-only PTQ can be applied to stored parameters.
- Activation quantization often needs representative calibration data.
- Stop if accuracy and performance targets are met.

# Static vs Dynamic Activation Quantization

## Static ranges

Calibration estimates activation ranges *before* deployment.

**Trade-off:** little runtime range-estimation cost; may miss input shifts.

## Dynamic ranges

Ranges are estimated from incoming activations *during* inference.

**Trade-off:** more adaptive; adds runtime reduction and scaling work.

Choose by model, hardware, and input variability.

# QAT and Precision-Sensitive Layers

## Quantization-aware training

- Simulate rounding/clipping in forward passes.
- Update parameters to tolerate the numerical disturbance.
- Export a low-precision inference graph.

## Decision

PTQ meets requirements? **Stop.**

Otherwise: better scaling, mixed precision or QAT.

Runtime support and measured quality determine the final choice.

# Why Might Some Weights Be Redundant?

- Overparameterized models can contain low-impact weights or units.
- **Pruning** removes weights or groups under a quality constraint.
- Remaining computations may be reorganized or fine-tuned.

## Two questions

**Statistical:** What can be removed without unacceptable quality loss?
**Systems:** Can the target runtime exploit what was removed?

Pruning | 2 min

# Magnitude-Based Pruning

## Simple rule

Select the weights with the smallest $|w_i|$ and set them to zero.

$$w = [-0.8,\ 0.03,\ 0.5,\ -0.02,\ 1.1]$$

$$\text{Prune two smallest} \longrightarrow [-0.8,\ 0,\ 0.5,\ 0,\ 1.1]$$

## Result

Two out of five entries are zero: **40% sparsity**.

# Worked Example: Prune a Weight Matrix

## Original and pruned (threshold $|w| < 0.1$)

$$W = \begin{bmatrix} .8 & .02 & -.4 \\ -.03 & .9 & .06 \\ .5 & -.01 & .2 \end{bmatrix} \quad \Rightarrow \quad W' = \begin{bmatrix} .8 & 0 & -.4 \\ 0 & .9 & 0 \\ .5 & 0 & .2 \end{bmatrix}$$

- Four of nine values become zero: $4/9 \approx 44.4\%$.
- Matrix shape remains $3 \times 3$.
- Dense storage still holds nine positions unless format changes.

# Sparsity and Compression Ratio

## Definitions

$$\text{sparsity} = \frac{\#\text{ zero entries}}{\#\text{ total entries}}$$

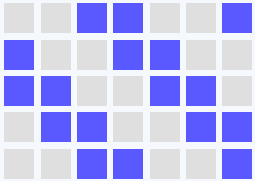
$$\text{raw parameter reduction} = \frac{N_{\text{before}}}{N_{\text{after}}}$$

## Important distinction

80% sparse *does not* imply 5× smaller on disk; sparse indices, pointers and encoding matter.

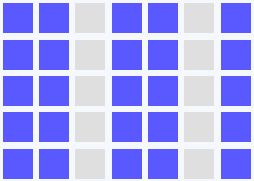
# Unstructured vs Structured Pruning

## Unstructured



Individual zeros: irregular execution.

## Structured



Entire columns / channels removed.

Regularity can turn sparsity into smaller dense operations.

# N:M Semi-Structured Sparsity

## 2:4 example

In each group of four weights, keep exactly two nonzeros.

$$[0.8,\ 0.02,\ -0.6,\ 0.01] \rightarrow [0.8,\ 0,\ -0.6,\ 0]$$

- Predictable local pattern supports specialized hardware paths.
- Logical sparsity $= 50\%$ (exact 2:4 pattern).
- Speedup requires explicit library and accelerator support.

# Fine-Tuning after Pruning




- One-shot pruning: a single large removal step.
- Iterative pruning: remove, recover, repeat.
- Recoverable quality loss is an empirical question.

# Reading the Pruning Objective

## Constrained idea

$$\min_W L(W) \quad \text{subject to} \quad \|W\|_0 \leq k$$

Few nonzero weights, while keeping prediction error small.

## A softer alternative

$$\min_W [L(W) + \lambda \|W\|_1]$$

Trade off prediction error and the sum of absolute weight magnitudes.

Interpret these goals; no optimization proof required.

# The 90%-Sparsity Paradox

**Claim to test**

"90% of weights are zero; inference must be 10× faster."

- Dense kernels still perform the original arithmetic.
- Sparse formats store indices / metadata.
- Irregular reads reduce efficiency.
- Other layers and launch overhead remain.

**Necessary test: benchmark real inference on target hardware.**

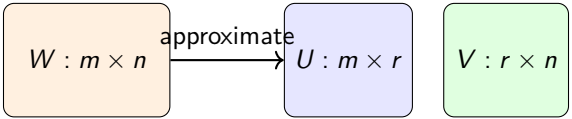
# Large Matrices Can Contain Redundant Structure

- Dense layer: $y = Wx$, $W \in \mathbb{R}^{m \times n}$.
- Some weight matrices are well approximated in a smaller-dimensional subspace.
- **Low-rank factorization** replaces one matrix with two thin factors.

**Question**

Can we preserve most useful behavior while reducing stored coefficients and arithmetic?

# Replace One Matrix with Two




$$W \approx UV, \qquad y \approx U(Vx)$$

$$\underbrace{mn}_{\text{original weights}} \longrightarrow \underbrace{mr + rn}_{\text{factor weights}}$$

# Worked Example: 4096-by-4096 at Rank 128

## Original dense matrix

$$N_{\text{original}} = 4096^2 = 16{,}777{,}216$$

## Two factors

$$N_{\text{factors}} = 4096(128) + 128(4096) = 1{,}048{,}576$$

$$\textbf{Theoretical reduction} = 16\times$$

Applies to idealized parameter and matrix-vector MAC counts, not measured latency.

# Rank Controls the Accuracy–Efficiency Trade-off

| Rank $r$ | Factor weights $r(4096 + 4096)$ | Ideal reduction |
|---|---|---|
| 64 | 524,288 | 32× |
| 128 | 1,048,576 | 16× |
| 512 | 4,194,304 | 4× |

- Lower rank: fewer weights, usually more approximation error.
- Higher rank: greater capacity, less savings.
- Validate task quality after factorization and fine-tuning.

# Low Rank: When Does It Help?

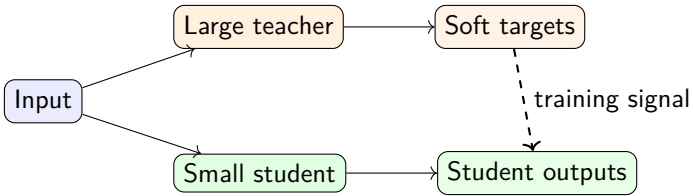
## Favors low rank

- Dense layers dominate stored weights or MACs.
- A small rank keeps quality acceptable.
- Two-factor kernels run efficiently on the target.

## Can disappoint

Extra launch, intermediate activations, approximation loss, and small original matrices.

# Train a Smaller Student from a Larger Teacher




- Train the compact student to reproduce useful teacher behavior.
- Deploy **only the student** for inference.

# Hard Labels vs Soft Predictions

| Signal | Cat | Dog | Fox |
|---|---|---|---|
| Ground-truth one-hot | 1.00 | 0.00 | 0.00 |
| Teacher probabilities | 0.85 | 0.10 | 0.05 |

- Hard label says which class is correct.
- Soft target also describes the teacher alternatives.
- Teacher behavior can carry class-similarity information.

# Temperature Softens the Distribution

## Temperature-scaled softmax

$$p_i^{(T)} = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- $z_i$: teacher or student logit.
- $T = 1$: ordinary softmax.
- Larger $T$: typically softer, more even probabilities.

Purpose: expose relative preferences among non-top classes.

# What Is the Distillation Loss Asking For?

## Intuition-first training objective

$$L_{\text{total}} = \alpha L_{\text{imitate}} + (1 - \alpha) L_{\text{labels}}$$

- $L_{\text{imitate}}$: student matches teacher probabilities.
- $L_{\text{labels}}$: student predicts ground-truth classes.
- $\alpha$: balance between the two goals.

*Optional notation: imitation often uses KL divergence, with temperature-dependent scaling.*

# Example: DistilBERT and the Cost Shift

## Published DistilBERT result (Sanh et al., 2019)

- About **40% fewer parameters** than BERT-base.
- About **60% faster** inference in the paper's evaluation.
- Retains about **97%** of BERT language-understanding performance as reported.

## Systems perspective

Extra teacher/student training cost now; smaller student inference cost across many requests later.

# Distillation Is Not Lossless Compression

## Can work well when

- Teacher is strong
- Student is adequate
- Distillation data matches deployment

## Failure modes

- Student lacks capacity
- Teacher errors transfer
- Domain shift
- Expensive training

**Always validate student quality and execution cost.**

# A Smaller Model Is Not Always Faster

## Latency components

$$T_{\text{endtoend}} \approx T_{\text{input}} + T_{\text{kernels}} + T_{\text{transfers}} + T_{\text{output}}$$

- Some time is outside the weight-heavy layers.
- Conversion and packing can add overhead.
- Compression must target the dominant bottleneck.

# Compute-Bound vs Memory-Bound

**Compute-bound**

Arithmetic units are the limiting resource.

**Promising:** fewer MACs or efficient low-precision instructions.

**Memory-bound**

Moving bytes is the limiting resource.

**Promising:** compact weights, cache reuse, fewer transfers.

Operational intensity = arithmetic work per byte moved.

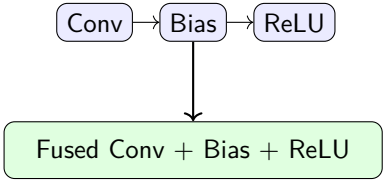
# Dense vs Sparse Execution

| Execution | Advantage | Typical cost |
|---|---|---|
| Dense GEMM | Very optimized / regular | Multiplies explicit zeros |
| Unstructured sparse | Skips selected nonzeros | Indices, irregular reads |
| Structured sparse | Regular constrained pattern | Hardware / kernel restrictions |
| Smaller dense model | Standard kernels | May lose more accuracy |

Data layout and kernel support

determine real sparsity benefits.

# Kernel Fusion and Execution Overhead




- Fusion can avoid intermediate memory traffic and launches.
- Factorization can introduce a new intermediate and extra launch.

# Precision Support and Accelerators

## What to verify before claiming acceleration

1. Does the device have efficient INT8 / INT4 / FP16 kernels?
2. Are the exact shapes and operators supported?
3. Is dequantization fused or performed separately?
4. Are activations, accumulators and sensitive ops supported?

INT4 weight storage does not imply native INT4 end-to-end execution.

# Benchmark the Model That Will Ship

## Minimum evaluation sheet

| | |
|---|---|
| Model quality | Accuracy / relevant task metric |
| Footprint | Model bytes, peak RAM |
| Performance | P50/P95 latency, throughput |
| Device | Exact accelerator, batch, runtime, precision |
| Operations | Warm-up, synchronization, input size |

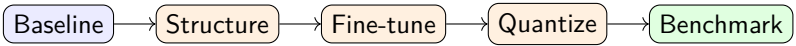
Compare under identical, representative conditions.

# Compare the Four Core Methods

| Method | What changes? | Key risk | Typical prerequisite |
|---|---|---|---|
| Quantization | Number representation | Error / unsupported low-bit kernels | Calibration or QAT |
| Pruning | Weights / structural groups | Accuracy and irregular sparsity | Sparse support or dense reshaping |
| Low rank | One matrix becomes two | Approximation / extra launch | Suitable layer and low rank |
| Distillation | Train a small student | Capacity and training cost | Teacher, data, training budget |

These methods target different forms of redundancy.

# Combine Techniques, Then Re-Benchmark




- Example path: structured pruning $\rightarrow$ fine-tuning $\rightarrow$ PTQ.
- Alternative: distill student $\rightarrow$ calibrate and quantize.
- Interactions can help or hurt; improvements need not multiply.

# Design Challenge: Edge Camera

## Given a measured baseline (illustrative)

- Accuracy: 92%; model: 120 MB.
- P95 latency: 85 ms/frame.
- Required: $\geq 90\%$ accuracy, $\leq 40$ MB, $\leq 45$ ms.

## In pairs: propose a first experiment

Choose one technique, name its main risk, and specify **what you measure** to decide whether to keep it.

# Three Takeaways: Structure, Precision, Hardware

1. **Structure:** remove, factor, or relearn unnecessary computation.
2. **Precision:** store and move numerical values economically.
3. **Hardware:** optimize the executable on the actual device.

**Compression is a measured systems trade-off.**

**Read:**
`https://mlsysbook.ai/vol1/10_model_compression/10_model_compression.html`

# Optional mathematical appendix: distillation objective

$$p_i^{(T)} = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

$$\mathcal{L} = \alpha T^2 D_{\mathrm{KL}}(p_{\mathrm{teacher}}^{(T)} \| p_{\mathrm{student}}^{(T)}) + (1-\alpha)\mathcal{L}_{\mathrm{CE}}$$

- KL divergence compares distributions; cross entropy uses true labels.
- The $T^2$ term is a common convention for balancing gradient magnitudes.
- Not required for the core lecture: no backpropagation derivation.

# References and instructor sources

- *Introduction to Machine Learning Systems*, Vol. I, Chapter 10: Model Compression. `https://mlsysbook.ai/vol1/10_model_compression/10_model_compression.html`
- Hinton, Vinyals and Dean (2015), *Distilling the Knowledge in a Neural Network*.
- Sanh et al. (2019), *DistilBERT, a distilled version of BERT*.
- Previous lecture styling and topic continuity adapted from the supplied `s-workflow-operations.tex`.